LABORATORY MANUAL

SC2103: Digital System Design

Experiment 1:

Simple Vending Machine

COLLEGE OF COMPUTING AND DATASCIENCE
NANYANG TECHNOLOGICAL UNIVERSITY

SC2103 Laboratory 1
Simple Vending Machine

Aim: The aim of this lab is to design, implement, and verify a Moore Finite State Machine (FSM) controller for a simple muesli bar vending machine using the Verilog Hardware Description Language (HDL) and validate the design through both behavioural simulation (in a testbench) and physical implementation on a Xilinx Artix-7 FPGA (Basys 3 board).

Objectives:

Task	Objective
Task 1	Design the **state diagram** for the vending machine controller, representing the logic for coin acceptance, vending, change rejection (overpayment), and refund/cancellation.
Task 2	Implement the FSM controller logic in **Verilog**, including defining states using parameter, implementing the synchronous state register, and writing the combinational logic for next state and outputs using an always block and a case statement.
Task 3	Develop a **Verilog testbench** to apply stimulus (input coins and cancel) to the FSM and verify that the FSM transitions between states and asserts the correct outputs (insert_coin, dispense, money_return) under various scenarios (vending, refund, invalid input).
Task 4	Synthesize and implement the verified Verilog design onto the target **FPGA**, including writing and applying a **XDC constraints file** to map the FSM's inputs (buttons) and outputs (LEDs) to the physical I/O pins of the Basys 3 board.

Finite State Machine (FSM) for a Muesli Bar Vending Machine

You will design an FSM controller for a vending machine that dispenses healthy muesli bars. The machine accepts only **50-cent** and **1-dollar** coins. One muesli bar cost **exactly $1**.

The machine works as follows:

- If the customer inserts **more than $1** (for example, 50c then $1), the FSM must **reject the transaction** and **return all coins**.
- At any time, if the **cancel** input is asserted, the FSM must **refund the amount inserted** and return to the default state.
- When a refund is issued, the FSM sends a **money_return** signal to the coin mechanism and must immediately return to the **default/reset state**.
- When the customer has inserted **exactly $1**, the FSM must **dispense the product immediately**.
- After dispensing, the machine will stay in the dispense state until a **manual reset** is applied. This reset simulates the signal normally provided by the vending mechanism.

Inputs: fifty – a single-cycle pulse indicating a 50-cent coin was inserted, dollar – a single-cycle pulse indicating a $1 coin was inserted, cancel – a single-cycle pulse requesting a refund, clk, rst.

Outputs: st – indicates the current state, insert_coin – enables coin acceptance, money_return – returns the inserted amount, dispense – triggers the vending mechanism

Task 1 (Pre-lab)

Sketch the state diagram that represents the above behaviour. You should use a Moore machine, where the outputs depend only on the current state. The most straightforward way is to think of the states as representing the amount inserted so far. The end state representing product vending should not have any transitions. We also assume only one input is asserted at any point in time. You should have 4 states in total (one of which is the vending end state). Verify your state diagram is identical to the one given in Fig. 1.

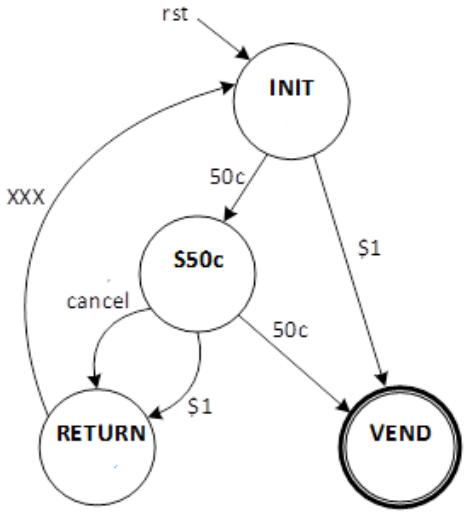


Fig. 1: State transition diagram for Simple Vending Machine

Using a text editor, implement the FSM in Verilog. Call it *lab1_FSM*. Your module should have five inputs and 4 outputs (including the state). See top_FSM.v, where the *lab1_FSM* module is used.

1. Declare the module and the port list. Declare current state (*st*) with sufficient wordlength.
2. Use the parameter keyword to define names for the four states and their assignment, as:

   parameter INIT=0, S50c=1, VEND=2, RETURN=3;
3. Declare the next state (*nst*) variable of sufficient wordlength.
4. Implement a synchronous always block to take the value of next state and assign it to the current state at the *clk* rising edge and add suitable reset behaviour (Note: *rst* is active high).
5. Implement the state transition and output logic. You may use a single combinational always block with a case statement. The case statement switches on the current state, and depending on the inputs will set a value for next state.

   When you have mutually exclusive inputs (as with a coin mechanism), where two inputs are not expected to be active (e.g. pressed) at the same time, it is better to use parallel ifs, as:

   if (a) nst = STA;
   if (b) nst = STB;

   rather than nested ifs, which forces the priority in the nested version and it consumes extra logic.

   if (a) nst = STA;
   else if (b) nst = STB;

   if (a) and (b) can never be true together, the priority logic is pointless.

Bring this *lab1_FSM.v* with you when you come for lab.

Task 2 (Implement the Design)

Start a new project in Vivado, as follows: (procedure is like SC1015- Digital logic lab 3, 4 and 5)

1. Start Vivado and create a new RTL project (Lab1) targeting the **Artix 7 xc7a35tcpg236-1 FPGA.**
2. Copy all the Verilog files from NTULearn zip file and place them in the same project location specified above (e.g. project_location/Lab5). Check the file extensions (e.g. name.v, name.xdc) are not modified.

3. Add the Verilog module *lab1_FSM* (Add Sources OR +, then select add or create design sources, then add). Ensure you include the timescale directive at the top of the Verilog file (`timescale 1ns / 1ps) (**OR** if you have not completed Task 1, create a new Verilog Module, called *lab1_FSM* with the required inputs and outputs, and then add the required functionality as in Task 1).
4. Correct any syntax errors by doing Synthesize in the Flow navigator window.
5. Add the constraints file, *Lab1.xdc*, (Add Sources OR +, Add or create constraints)

Task 3 (FSM Testbench)

Rather than load a design straight onto the FPGA, it is prudent to test it in a simulator first. Here we will build a Verilog testbench for the vending machine.

1. Create a new Verilog Test Fixture called *lab1_FSM_tb* (Add Sources OR +. You should see a bare testbench. You will need to instantiate your module, name the instance DUT, and then declare the **reg** and **wire** signals. Add an empty initial block.
2. Add a clock **always** block to toggle every 5 timesteps (after the **initial** block).
   always #5 clk = ~clk;
3. Inside the **initial** block, add the following testbench code to initialise the inputs and test different functions of the state machine. This code sets reset to 0 (de-asserted) after 10 timesteps. It also tests for different functionality, including: vending, cancelling a transaction and money return. Since the clock has a 10 timestep period, you should wait that long (#10) before issuing new sets of inputs.

```
// Add stimulus here
clk = 0; rst = 1; fifty = 0; dollar = 0; cancel = 0;
#10 rst = 0;                // to INIT (0) state
#10 fifty = 1;              // to S50c (1) state
#10 fifty = 0;
#10 fifty = 1;              // to VEND (2) state
#10 fifty = 0;
#20 rst = 1;                // RESET, to INIT (0) state
#10 rst = 0;
#10 dollar = 1;             // to  VEND (2) state
#10 dollar = 0;
#20 rst = 1;                // RESET, to INIT (0) state
#10 rst = 0;
#10 fifty = 1;              // to S50c (1) state
#10 fifty = 0;
#10 dollar = 1;             // to RETURN (3) state
#10 dollar = 0;             // to INIT (0) state
#20 fifty = 1;              // to S50c (1) state
#10 fifty = 0;
#10 cancel = 1;             // to RETURN (3) state
#10 cancel = 0;             // to INIT (0) state
```

Note: You only need to assert or de-assert signals that change.

After another 20 timesteps, add a **$finish()** statement to end the simulation.

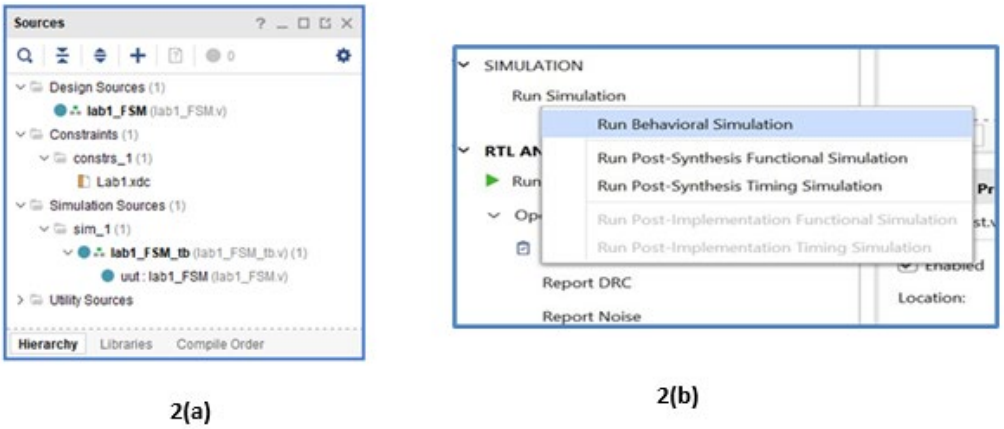
Check that the sources hierarchy (in the PROJECT MANAGER, Sources window) is like the Fig.2(a). If it is not, fix or check with the lab tutor.

4. Select the file for simulation under simulation sources (here lab1_FSM_tb) and then on the ***Flow Navigator*** (left hand panel) click **Run Simulation > Run Behavioral Simulation** to launch the simulator as shown in Fig. 2(b).

5. On the simulation window, click the **Zoom Fit (**  **)** button to get a good view of the simulation result.

6. In the waveform window if we want to add the next state (*nst*) instance (in Scope, expand the

lab1_FSM_tb instance and then click on DUT. In the objects window drag and drop n*st[1:0]* into the name field of the waveform window). Restart and rerun the simulation and check that you obtain the expected values by looking at the wave window. Try other inputs, including invalid ones and see what happens.



2(a) 2(b)

Fig. 2(a): Snapshot of sources attached to the project, Fig. 2(b) Simulation

Task 4 (Implement on the FPGA)

Implement the FSM on the Basys3 board as shown in Fig. 3 (below).

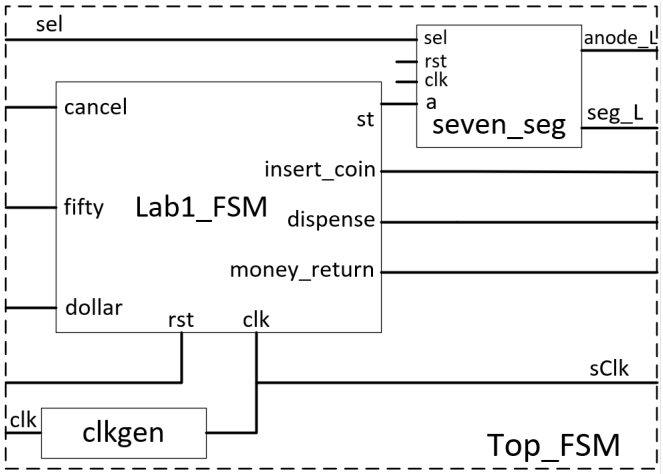


Fig. 3: Module instantiation in Top_FSM

- You are given the top level module (*top_FSM.v*) which instantiates 2 extra modules (*clockgen.v*, which slows the system clock from 100MHz to about 0.4Hz; and *seven_seg.v*, which displays the current state).

  Check point: Show the synthesized diagram to the TA (Lab performance marks)

  Constraints file setup, downloading bitstream and showcasing the finite state machine on the board.

1. You will also need the *Lab1.xdc* file to map the top-level module inputs and outputs to the Basys3 FPGA board.

2. *Lab1.xdc* already has the *clk*, *rst (btnU T18 pin)* and *sel* (btnD U17 pin) signals mapped. (check Lab1.xdc to understand the same)
3. You will need to add *fifty* (Pin W19), *dollar* (Pin U18), and *cancel* (Pin T17) to the 3 horizontal push button switches. Refer to Fig. 4.

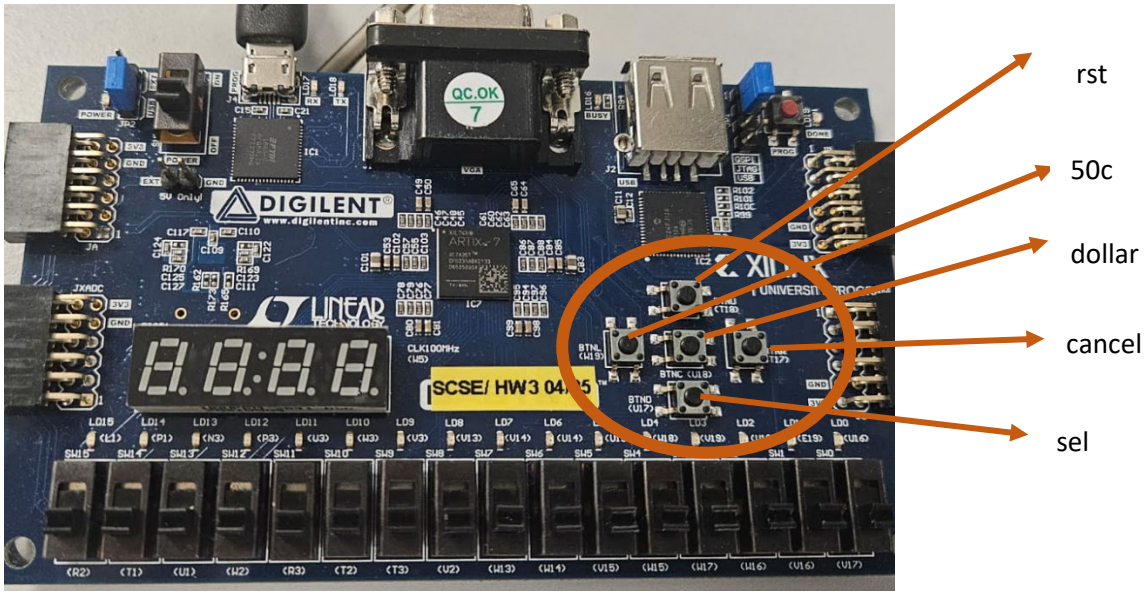


Fig. 4: Basys3 FPGA board showing the pin mappings for inputs

4. You will also need to map *insert_coin*, *dispense* and *money_return* (to the LEDs: LD0 to LD2) as shown in Fig. 5.

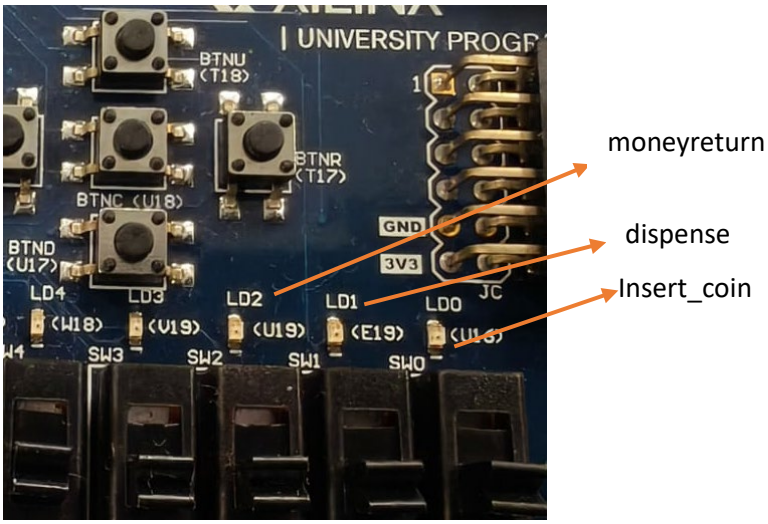


Fig.5: Basys3 FPGA board showing the pin mappings for outputs

5. Note that *sClk* has already been mapped to LED LD7 for you. You need to do this for both the location (PACKAGE_PIN) and the I/O standard (IOSTANDARD).
6. In file *seven_seg.v*, edit the bench number, *benchNo_Hi* and *benchNo*, (above the **endmodule** statement) to reflect **your** bench number. It is currently set at 95. The 7-segment outputs have been mapped for you.
7. **For generating bitstream:** You need to do Synthesise, implementation and Generate the bitstream (checking and fixing any applicable errors and warnings at each step) of the top verilog module *Top_FSM*.
8. Check that the Basys 3 board connected to the computer via a USB cable with the red power LED lit. (You may have to switch on the board)

9. Open the hardware Manager in vivaldo as shown in Fig. 6(a). Open Target and select Auto Connect. Verify the Digilent device is shown in the Hardware window as indicated in Fig. 6(b). Then select Program Device and click Program as shown in Fig. 6(c).

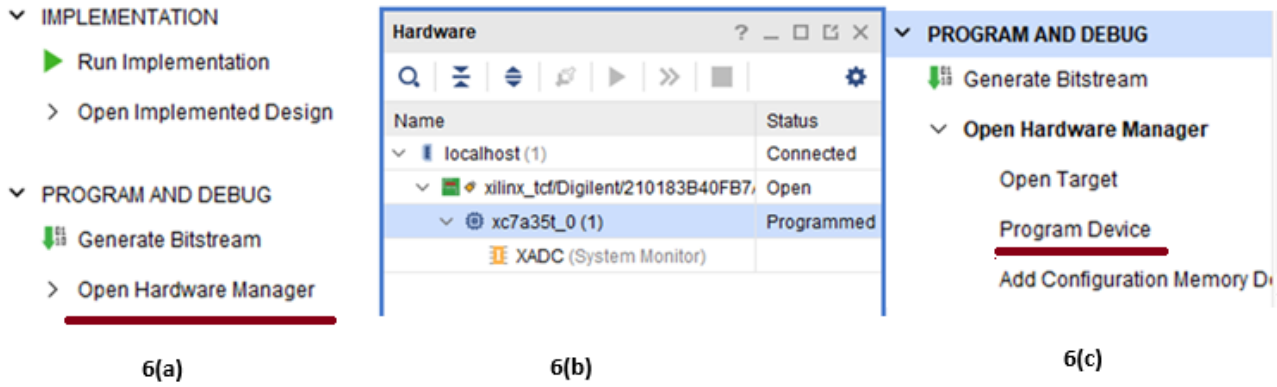


Fig. 6(a): Hardware manager (Vivaldo), Fig. 6(b): Digilent device in the Hardware window, Fig. 6(c) How to program device

10. Verify that the FPGA implementation works as expected. **NOTE: You may have to press the buttons for some time due to the slow clock. They need a rising edge of sClk.** You should also try some invalid inputs (such as pressing the fifty and dollar buttons at the same time while in state 0 or pressing the fifty and cancel buttons at the same time while in state 1) just to see what happens.

Note: The following warnings are OK

Synthesis:

WARNING: [Synth 8-3917] design top_FSM has port anode_L[3] driven by constant 1 (See note below)
WARNING: [Synth 8-3917] design top_FSM has port anode_L[2] driven by constant 1 (See note below)
WARNING: [Constraints 18-5210] No constraints selected for write.

Implementation:

WARNING: [Place 46-29] place_design is not in timing mode. Skip physical synthesis in placer

Bitstream:

There should be NO warnings

Notes:

The two MSBs of *anode_L* (in seven_seg.v) are not used. Fixed High.
The other warnings are system warnings and can be ignored.

Any other warnings will probably need to be corrected.